Chapter 12 — Testing React

A test is a small program that uses your code and checks the result, so that a computer — not you, clicking around — notices when something breaks. For React components the most useful tests **render the component, interact with it the way a user would, and check what the user would see**. That is the idea behind **React Testing Library** (RTL), the standard tool for component tests, and it's what this chapter teaches.

General testing ideas — what a test is, `node:test` (the test runner built into Node.js), assertions (checks that fail the test if a condition is false), test doubles and mocking (stand-in fakes that replace real parts of the system during a test) — are covered in the JavaScript Handbook (Chapter 16). Here we apply them to React.

What you'll learn

#	Topic	You will be able to…
12.1	What to test, and the Testing Library philosophy	decide what a good React test checks, and what it ignores
12.2	Your first component test	render, find, interact, assert — and clean up
12.3	Queries and their priority	choose `getByRole` first, and know `get`/`query`/`find` and `All` variants
12.4	`user-event` — interacting like a user	type, click, select, tab and press keys realistically
12.5	Async tests	wait for UI with `findBy`, `waitFor`, `waitForElementToBeRemoved`
12.6	Mocking `fetch` (and when to use MSW)	test loading, success, HTTP errors, network errors and races
12.7	Testing custom Hooks	use `renderHook`, `act`, fake timers — or a test component
12.8	Providers, routers and custom render	test components that need context or a router
12.9	Setting up Vitest or Jest in a real project	configure jsdom, jest-dom matchers and scripts
12.10	Good tests, bad tests, and the bigger picture	avoid brittle tests; where E2E and other tests fit

Code: `code/`. Every file is a **runnable test file**: React 19.3, React Testing Library 16.3, `@testing-library/user-event` 14.6 and jsdom (a pure-JavaScript imitation of the browser's DOM — the Document Object Model, the tree of elements that makes up a page — that runs in Node.js, the program that runs JavaScript outside a browser), executed by Node's built-in test runner (`node:test`) with `node:assert/strict` for assertions. The output shown is Node's `spec` reporter (a reporter is the part of the test runner that formats the results; `node --test-reporter=spec file.mjs`); times in parentheses vary per run, and the summary lines for suites (named groups of related tests), cancelled, skipped and todo (all 0) are left out. Real projects usually run the same tests with Vitest or Jest (the two most popular JavaScript test runners: programs that find test files, run them and report the results) — §12.9 shows that setup (not run here).

12.1 What to test, and the Testing Library philosophy

What it is

React Testing Library is a small set of helpers built on **DOM Testing Library**: `render` a component into a real (simulated) DOM, **query** elements the way a person finds them (by role, label, text), fire **events**, and assert on the resulting DOM. Its guiding principle, in short: the closer a test is to how people actually use the software, the more you can trust it when it passes.

In practice that means:

Test this (what users experience)	Not this (implementation details — how the code works inside)
The heading shows "Ada Lovelace" after loading	The component's `user` state variable
Clicking **Add to cart** updates the badge to 2	That `dispatch` was called with `{ type: 'add' }`
The **Submit** button is disabled until terms are accepted	Which Hook or CSS class name is used
An error message appears when the server returns 404 (the HTTP "not found" status)	The number of re-renders

Analogy: a restaurant inspector who orders food and tastes it, rather than reading the chef's recipe cards. If the chef reorganises the kitchen (a refactor — restructuring code without changing what it does) but the food is the same, the inspection still passes — and it fails exactly when the food gets worse.

Why it matters

- **Refactor-proof**: tests that don't touch internals keep passing when you rename state, split components, switch from `useState` to `useReducer`, or enable React Compiler (Chapter 10).
- **Accessibility for free** (accessibility means the UI works for everyone, including people using a screen reader — software that reads the page aloud — or only a keyboard): finding elements by role and accessible name only works if the UI is accessible (you'll see a test catch an unlabelled icon button in §12.3).
- **Confidence**: a test that clicks the real button through the real event system catches wiring bugs (mistakes in how the pieces are connected, e.g. a handler never attached to the button) that a unit test of a handler function would miss.

Where component tests sit

Kind	Checks	Tools	Speed
Unit	one function (a reducer, a formatter)	`node:test`, Vitest, Jest	fastest
Component / integration	one component or a feature of several, in a simulated DOM	RTL + jsdom (this chapter)	fast
End-to-end (E2E)	the whole app in a real browser against a real or staged backend (a test copy of the server)	Playwright, Cypress (tools that drive a real browser)	slowest

A common strategy (sometimes called the "testing trophy") puts most effort into **integration-style component tests**, a few E2E tests for critical flows (sign-up, checkout), and unit tests for tricky pure logic (code whose output depends only on its inputs, with no side effects such as network calls) — plus static checks (checks that read the code without running it: TypeScript, Chapter 11; ESLint, a linter that flags likely bugs and bad patterns) that catch whole classes of mistakes without running anything.

> **TL;DR**
>
> Render, find like a user, act like a user, assert on what a user sees. Don't test state, props, or which functions were called internally.

12.2 Your first component test

How it works

1. **Arrange** — `render(<Counter />)` mounts the component (creates its DOM and attaches it to the page) into `document.body` (jsdom provides the DOM in Node).

2. **Act** — find an element with a query (`screen.getByRole('button', { name: 'Increment' })`) and interact (`fireEvent.click(...)` or `user-event`, §12.4).
3. **Assert** — check the DOM: text, disabled state, presence or absence of elements.
4. **Clean up** — unmount (remove the component and its DOM) after each test so tests don't see each other's DOM. With Jest or Vitest (with `globals: true`) RTL does this automatically; with `node:test` you register `afterEach(cleanup)` yourself.

`screen` is an object with every query bound to `document.body`, so you don't need to keep the return value of `render`.

`code/01_first_test.jsx`:

```
import '../../../../tools_dom.mjs';                    // jsdom globals (window, document) for Node
import { test, afterEach } from 'node:test';
import assert from 'node:assert/strict';
import { useState } from 'react';
import { render, screen, fireEvent, cleanup } from '@testing-library/react';

function Counter({ initial = 0, step = 1 }) {
  const [count, setCount] = useState(initial);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(c => c + step)}>Increment</button>
      <button onClick={() => setCount(initial)} disabled={count === initial}>Reset</button>
    </div>
  );
}

afterEach(cleanup);

test('shows the initial count', () => {
  render(<Counter initial={5} />);
  assert.ok(screen.getByText('Count: 5'));                              // getBy* throws if not found
});

test('increments by step when the button is clicked', () => {
  render(<Counter step={10} />);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 20');
});

test('Reset is disabled until the count changes', () => {
  render(<Counter />);
  const reset = screen.getByRole('button', { name: 'Reset' });
  assert.equal(reset.disabled, true);
  fireEvent.click(screen.getByRole('button', { name: 'Increment' }));
  assert.equal(reset.disabled, false);
  fireEvent.click(reset);
  assert.equal(screen.getByText(/Count:/).textContent, 'Count: 0');
});
```

Output:

```
✔ shows the initial count (13.84825ms)
✔ increments by step when the button is clicked (50.187958ms)
✔ Reset is disabled until the count changes (3.544833ms)
ℹ tests 3
ℹ pass 3
ℹ fail 0
ℹ duration_ms 70.571583
```

What about `act()`?

React batches updates (groups several state changes into one re-render) and runs effects (code in `useEffect` that runs after rendering) asynchronously (a little later, not in the same step). `act(() => { ... })` tells React "apply everything this callback caused before I look at the DOM". RTL's `render`,

`fireEvent`, `user-event` actions and async utilities already wrap themselves in `act`, so in component tests you rarely write it. You do need it when you trigger state updates **directly**, e.g. calling a Hook's function in `renderHook` (§12.7) or advancing fake timers. The warning *"An update to X inside a test was not wrapped in act(...)"* usually means something finished **after** your assertions — await it with `findBy`/`waitFor` (§12.5).

With jest-dom (in Vitest/Jest)

Projects usually add `@testing-library/jest-dom`, which gives readable matchers (assertion helpers such as `toBeDisabled()`) with good failure messages: `expect(reset).toBeDisabled()`, `expect(el).toHaveTextContent('Count: 20')`, `expect(el).toBeInTheDocument()`. This book's runner uses plain `assert`, which checks the same DOM properties (`.disabled`, `.textContent`).

> **TL;DR**
>
> `render` → `screen.getByRole(...)` → interact → assert on the DOM; `cleanup` after each test (automatic in Jest/Vitest).

12.3 Queries and their priority

Query variants

Every query comes in six forms. The prefix decides **what happens when zero or several elements match**, and whether it waits:

Variant	0 matches	1 match	2+ matches	Waits?	Use it to…
`getBy…`	throws	returns element	throws	no	find something that must be there **now**
`queryBy…`	returns `null`	returns element	throws	no	assert something is **absent**
`findBy…`	rejects	resolves element	rejects	**yes** (retries, default 1000 ms)	wait for something to **appear**
`getAllBy…`	throws	array	array	no	find several
`queryAllBy…`	`[]`	array	array	no	count, possibly zero
`findAllBy…`	rejects	array	array	**yes**	wait for several

`findBy…` returns a **Promise** — an object standing for a result that arrives later. It *resolves* with the element when found, or *rejects* with an error after the timeout, so you `await` it.

Which query to use (priority order)

Testing Library recommends choosing queries in this order — the higher, the closer to how people (including screen-reader users) find things:

1. **Accessible to everyone**
    1. `ByRole` — elements by their ARIA role (ARIA, Accessible Rich Internet Applications, is the set of attributes that tell assistive tools what an element is; its *role* is the kind of thing it is: button, link, heading, textbox, checkbox, list, …) and **accessible name** (`{ name: 'Save' }`). Use this for almost everything.
    2. `ByLabelText` — form fields by their `<label>`.
    3. `ByPlaceholderText` — only if a field truly has no label (a placeholder is not a label).
    4. `ByText` — non-interactive text (paragraphs, spans, divs).
    5. `ByDisplayValue` — a form field by its current value.
2. **Semantic attributes**

    1. `ByAltText` — images and other elements with `alt`.
    2. `ByTitle` — the `title` attribute (not reliably announced by screen readers).
3. **Test IDs**: `ByTestId` — `data-testid="…"`, only when nothing user-visible identifies the element.

The **accessible name** is what assistive technology (screen readers, voice control and similar tools) announces: a button's text, an input's label, an `aria-label` or `aria-labelledby`, an image's `alt`. Text matchers can be exact strings (`'Save'`), case-insensitive regexes (regular expressions, patterns for matching text: `/save/i`), or functions.

`code/02_queries.jsx` (component and key tests):

```
function ProductCard({ product, inCart }) {
  return (
    <article aria-labelledby="p-title">
      <img src={product.image} alt={`Photo of ${product.name}`} />
      <h2 id="p-title">{product.name}</h2>
      <p>{product.price.toFixed(2)} EUR</p>
      <label htmlFor="qty">Quantity</label>
      <input id="qty" type="number" defaultValue={1} />
      <input placeholder="Gift message (optional)" />
      <ul aria-label="features">{product.features.map(f => <li key={f}>{f}</li>)}</ul>
      <button>Add to cart</button>
      <button title="Save for later">♡</button>
      {inCart && <p role="status">Already in your cart</p>}
      <span data-testid="sku">{product.sku}</span>
    </article>
  );
}

test('priority 1: queries everyone can perceive (role, label, placeholder, text, display value)', () => {
  render(<ProductCard product={lamp} />);
  assert.equal(screen.getByRole('heading', { name: 'Desk lamp', level: 2 }).tagName, 'H2');
  assert.equal(screen.getByRole('button', { name: 'Add to cart' }).tagName, 'BUTTON');
  assert.equal(screen.getByRole('article', { name: 'Desk lamp' }).tagName, 'ARTICLE');  // name from aria-
labelledby
  assert.equal(screen.getByLabelText('Quantity').value, '1');
  assert.ok(screen.getByPlaceholderText(/gift message/i));
  assert.ok(screen.getByText('39.50 EUR'));
  assert.equal(screen.getByDisplayValue('1').id, 'qty');
});

test('priority 2: semantic attributes (alt text, title)', () => {
  render(<ProductCard product={lamp} />);
  assert.equal(screen.getByAltText(/photo of desk lamp/i).getAttribute('src'), '/lamp.jpg');
  assert.ok(screen.getByTitle('Save for later'));
  // A button's accessible name comes from its CONTENT first; title is only a fallback. So a screen
  // reader announces this icon button as "♡" - the test just revealed an accessibility bug.
  assert.ok(screen.getByRole('button', { name: '♡' }));
  assert.equal(screen.queryByRole('button', { name: 'Save for later' }), null);
  // Fix in the component: <button aria-label="Save for later">♡</button>
});

test('getAllBy for many matches, within() to search inside one element', () => {
  render(<ProductCard product={lamp} />);
  const list = screen.getByRole('list', { name: 'features' });
  const items = within(list).getAllByRole('listitem');
  assert.deepEqual(items.map(li => li.textContent), ['LED', 'Dimmable', 'USB-C']);
});

test('queryBy to assert that something is NOT there', () => {
  const { rerender } = render(<ProductCard product={lamp} inCart={false} />);
  assert.equal(screen.queryByRole('status'), null);         // queryBy returns null instead of throwing
  rerender(<ProductCard product={lamp} inCart />);
  assert.equal(screen.getByRole('status').textContent, 'Already in your cart');
});
// ... plus: a failing getBy's message, logRoles, and getByTestId('sku')
```

Output:

```
    error message (first 3 lines):
    | Unable to find an accessible element with the role "button" and name "Buy now"
    |
    | Here are the accessible roles:
    logRoles output:
    | list:
    | Name "tiny":
    | <ul
    |   aria-label="tiny"
    | />
    | --------------------------------------------------
    | listitem:
    | Name "":
    | <li />
    | --------------------------------------------------
✔ priority 1: queries everyone can perceive (role, label, placeholder, text, display value) (61.198666ms)
✔ priority 2: semantic attributes (alt text, title) (6.331375ms)
✔ priority 3: test IDs, only when nothing user-visible identifies the element (2.198959ms)
✔ getAllBy for many matches, within() to search inside one element (8.569083ms)
✔ queryBy to assert that something is NOT there (4.234708ms)
✔ a failing getBy explains itself (13.676958ms)
✔ logRoles prints every role and accessible name - handy when writing queries (2.698125ms)
ℹ tests 7
ℹ pass 7
ℹ fail 0
```

Notes on the output

- A failing `getBy` lists **every accessible role and name** on the page (cut to 3 lines here), followed by the DOM. Reading it usually tells you immediately whether you used the wrong name or the element really is missing.
- `logRoles(container)` prints the same role list on demand; `screen.debug()` prints the current DOM. Both are for writing tests, not for leaving in them.
- Roles you'll use most: `button`, `link`, `heading` (with `level`), `textbox`, `checkbox`, `radio`, `combobox` (a `<select>`), `spinbutton` (a number input), `list`/`listitem`, `table`/`row`/`cell`, `dialog`, `alert`, `status`, `img`, `navigation`, `main`.
- The icon-button test shows RTL doing accessibility work: the button had a `title` but its announced name was "♡". Queries by role make such bugs visible.

Common mistakes

Mistake	Better
`container.querySelector('.btn-primary')`	`screen.getByRole('button', { name: 'Save' })`
`getByTestId` everywhere	role/label/text first; test IDs only as a last resort
`expect(queryByText('x')).toBeTruthy()` to check presence	`getByText` (throws with a helpful message)
`getByText` to assert absence (it throws)	`queryByText(...) === null`
`getBy` for things that appear later	`findBy` (§12.5)
Exact long text strings that break on copy edits	a regex for the stable part: `/order confirmed/i`

TL;DR

Prefer `getByRole(role, { name })`; then label, placeholder, text, display value; then alt/title; test IDs last. `get` = must exist now, `query` = may be absent, `find` = wait for it.

12.4 `user-event` — interacting like a user

What it is

`fireEvent` dispatches (sends) **one** DOM event. A real user typing "hi" produces focus (the event when an element becomes the one receiving keyboard input), click, and for each key `keydown`, `keypress`, `input`, `keyup` — and a real click on a disabled button does nothing, a real Tab moves focus in document order. `@testing-library/user-event` simulates those full interactions, so your tests exercise the same paths as real users (including bugs that only appear with the full sequence, like a handler on `keydown`).

```
import userEvent from '@testing-library/user-event';
const user = userEvent.setup();            // v14 API: create a session, then await each action
await user.type(input, 'Ada');
await user.click(button);
await user.selectOptions(select, 'pro');
await user.keyboard('{Enter}');            // special keys in braces: {Enter} {Escape} {Backspace} {Shift>}...
await user.tab();
await user.clear(input);
await user.upload(fileInput, file);
await user.hover(element);
```

All methods are **async** (they return a Promise and finish later) — always `await` them.

`code/03_user_event.jsx`:

```
function SignupForm({ onSubmit }) {
  const [form, setForm] = useState({ name: '', plan: 'free', terms: false });
  const set = (k, v) => setForm(f => ({ ...f, [k]: v }));
  return (
    <form onSubmit={e => { e.preventDefault(); onSubmit(form); }}>
      <label>Name <input value={form.name} onChange={e => set('name', e.target.value)} /></label>
      <label>Plan
        <select value={form.plan} onChange={e => set('plan', e.target.value)}>
          <option value="free">Free</option><option value="pro">Pro</option>
        </select>
      </label>
      <label><input type="checkbox" checked={form.terms} onChange={e => set('terms', e.target.checked)} /> I
accept the terms</label>
      <button type="submit" disabled={!form.terms || !form.name}>Create account</button>
    </form>
  );
}

test('fireEvent vs userEvent: which DOM events does "typing" produce?', async () => {
  render(<input aria-label="box" />);
  const box = screen.getByLabelText('box');
  const seen = [];
  for (const type of ['focus', 'keydown', 'keypress', 'input', 'keyup', 'click'])
    box.addEventListener(type, () => seen.push(type));
  fireEvent.change(box, { target: { value: 'hi' } });
  console.log('    fireEvent.change   ->', seen.join(', ') || '(no listened-to events; only a synthetic change)');
  seen.length = 0; box.value = '';
  const user = userEvent.setup();
  await user.type(box, 'hi');
  console.log('    user.type(box, hi) ->', seen.join(', '));
});

test('fills in and submits the whole form like a user', async () => {
  const user = userEvent.setup();
  const submitted = [];
  render(<SignupForm onSubmit={data => submitted.push(data)} />);
  const button = screen.getByRole('button', { name: 'Create account' });
  assert.equal(button.disabled, true);
  await user.type(screen.getByRole('textbox', { name: 'Name' }), 'Ada');
  await user.selectOptions(screen.getByRole('combobox', { name: 'Plan' }), 'pro');
  await user.click(screen.getByRole('checkbox', { name: 'I accept the terms' }));
  assert.equal(button.disabled, false);
  await user.click(button);
  assert.deepEqual(submitted, [{ name: 'Ada', plan: 'pro', terms: true }]);
});

test('keyboard only: Tab moves focus in DOM order, Enter submits', async () => {
  const user = userEvent.setup();
  const submitted = [];
  render(<SignupForm onSubmit={data => submitted.push(data)} />);
  await user.tab();
  assert.equal(document.activeElement, screen.getByRole('textbox', { name: 'Name' }));
  await user.keyboard('Grace');
  await user.tab(); await user.tab();                    // skip the select, reach the checkbox
  assert.equal(document.activeElement, screen.getByRole('checkbox'));
  await user.keyboard(' ');                              // Space toggles a checkbox
  await user.click(screen.getByRole('textbox', { name: 'Name' }));
  await user.keyboard('{Enter}');                        // Enter in a text field submits the form
  assert.deepEqual(submitted, [{ name: 'Grace', plan: 'free', terms: true }]);
});

test('clicking a disabled button does nothing', async () => {
  const user = userEvent.setup();
  let clicks = 0;
  render(<button disabled onClick={() => clicks++}>Pay</button>);
  await user.click(screen.getByRole('button', { name: 'Pay' }));
  assert.equal(clicks, 0);
});
```

Output:

```
    fireEvent.change   -> (no listened-to events; only a synthetic change)
    user.type(box, hi) -> focus, click, keydown, keypress, input, keyup, keydown, keypress, input, keyup
✔ fireEvent vs userEvent: which DOM events does "typing" produce? (72.232958ms)
✔ fills in and submits the whole form like a user (55.993083ms)
✔ keyboard only: Tab moves focus in DOM order, Enter submits (49.1405ms)
✔ clicking a disabled button does nothing (7.998ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

`fireEvent` **vs** `user-event`

	`fireEvent`	`user-event`
What it does	dispatches exactly one event	simulates the full interaction (focus, pointer, keyboard, input)
Sync/async	sync	async (`await`)
Respects disabled, `pointer-events: none`, focus order	no	yes
When to use	events user-event doesn't model (e.g. `scroll`, custom events), or very low-level tests	**default** for anything a user does

The keyboard test is also an accessibility check: it proves the form is usable without a mouse.

TL;DR

`const user = userEvent.setup()` then `await user.type/click/selectOptions/keyboard/tab(...)`. Prefer it over `fireEvent` for user actions.

12.5 Async tests

The problem it solves

Real components load data, wait for timers and animate. Right after `render` the data isn't there yet, so a `getBy` fails. You must **wait for the UI**, not for a fixed amount of time (`await sleep(500)` is slow and flaky — a flaky test sometimes passes and sometimes fails with no code change; it passes on your machine and fails on a busy CI server, the continuous-integration machine that runs tests automatically on every push).

Tools

Tool	Waits for	Typical use
`await screen.findByRole(...)`	an element to appear (retries a `getBy` query)	data loaded, dialog opened
`await waitFor(() => { ...assertions... })`	a callback to stop throwing	anything that isn't "an element appears": a text change, a mock being called
`await waitForElementToBeRemoved(() => screen.queryByText('Loading…'))`	an element to disappear	spinners (animated loading indicators), closing dialogs

All of them retry every 50 ms (default) until success or a timeout (default 1000 ms), and wrap updates in `act` for you.

`code/04_async.jsx`:

```
const api = {
  loadOrders: () => new Promise(r => setTimeout(() => r([{ id: 1, total: 20 }, { id: 2, total: 35 }]), 50)),
  save: () => new Promise(r => setTimeout(r, 30)),
};

function Orders() {
  const [orders, setOrders] = useState(null);
  const [saved, setSaved] = useState(false);
  useEffect(() => { api.loadOrders().then(setOrders); }, []);
  if (!orders) return <p>Loading orders…</p>;
  return (
    <>
      <ul>{orders.map(o => <li key={o.id}>Order {o.id}: {o.total} EUR</li>)}</ul>
      <button onClick={async () => { await api.save(); setSaved(true); }}>Save</button>
      {saved && <p role="status">Saved!</p>}
    </>
  );
}

test('right after render only the loading state exists', () => {
  render(<Orders />);
  assert.ok(screen.getByText('Loading orders…'));
  assert.equal(screen.queryByText(/Order 1/), null);
});

test('findBy waits (retries up to 1000 ms by default) until the element appears', async () => {
  render(<Orders />);
  const items = await screen.findAllByRole('listitem');
  assert.deepEqual(items.map(li => li.textContent), ['Order 1: 20 EUR', 'Order 2: 35 EUR']);
  assert.equal(screen.queryByText('Loading orders…'), null);
});

test('waitForElementToBeRemoved waits for the spinner to go away', async () => {
  render(<Orders />);
  await waitForElementToBeRemoved(() => screen.queryByText('Loading orders…'));
  assert.equal(screen.getAllByRole('listitem').length, 2);
});

test('waitFor retries any assertion until it passes', async () => {
  const user = userEvent.setup();
  render(<Orders />);
  await user.click(await screen.findByRole('button', { name: 'Save' }));
  await waitFor(() => assert.equal(screen.getByRole('status').textContent, 'Saved!'));
});

test('findBy gives up after its timeout with a clear error', async () => {
  render(<Orders />);
  const t0 = Date.now();
  await assert.rejects(screen.findByText('Order 99', {}, { timeout: 200 }), /Unable to find an element with the
text: Order 99/);
  console.log(`    rejected after ~${Math.round((Date.now() - t0) / 100) * 100} ms`);
});
```

Output:

```
✔ right after render only the loading state exists (14.355875ms)
✔ findBy waits (retries up to 1000 ms by default) until the element appears (98.487708ms)
✔ waitForElementToBeRemoved waits for the spinner to go away (55.889ms)
✔ waitFor retries any assertion until it passes (101.515625ms)
    rejected after ~200 ms
✔ findBy gives up after its timeout with a clear error (202.66725ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Rules for `waitFor`

- Put **only assertions** inside the callback — it may run many times. Do actions (clicks, typing) outside it.

- One thing to wait for per `waitFor`; then assert the rest normally.
- Prefer `findBy` when you're waiting for an element; it reads better and gives better errors.
- Don't wrap `findBy` in `waitFor` — it already waits.

Common mistakes

- Forgetting `await` before `findBy`/`waitFor`/`user.*` — the test finishes before the UI updates, passes by accident, and prints an `act(...)` warning.
- Arbitrary `sleep`s. Wait for a **condition** instead. (For code that genuinely depends on time — debounce (wait until input has stopped for a moment, then act once), polling (asking the server again every few seconds) — use fake timers, §12.7.)

TL;DR

`await findBy…` for appearing elements, `waitForElementToBeRemoved` for disappearing ones, `waitFor(() => assertion)` for anything else. Never sleep.

12.6 Mocking `fetch` (and when to use MSW)

What it is

A **mock** replaces a real dependency (something your code relies on, such as the network or another module) with a controllable fake. Components that call `fetch` shouldn't hit a real server in tests: it's slow, needs a network, and returns different data over time. Mocking `fetch` lets each test choose the response — success, an HTTP error (HTTP is the request/response protocol browsers and servers use; an error is a response with a 4xx/5xx status code — see System Design Fundamentals), a network failure, a slow reply — and check the UI for each.

With `node:test`, `mock.method(globalThis, 'fetch', impl)` replaces the global (available everywhere, attached to `globalThis`) `fetch` and records every call; `mock.restoreAll()` puts the original back. (Vitest: `vi.spyOn(globalThis, 'fetch').mockResolvedValue(...)`; Jest: `jest.spyOn(global, 'fetch')`.) Node 18+ has `Response` built in, so the fake can return a real `Response` object.

`code/05_mocking_fetch.jsx`:

```
function UserProfile({ userId }) {
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const controller = new AbortController();
    setState({ status: 'loading' });
    fetch(`/api/users/${userId}`, { signal: controller.signal })
      .then(res => { if (!res.ok) throw new Error(`Server said ${res.status}`); return res.json(); })
      .then(user => setState({ status: 'done', user }))
      .catch(err => { if (err.name !== 'AbortError') setState({ status: 'error', message: err.message }); });
    return () => controller.abort();                  // cancel if userId changes or we unmount
  }, [userId]);

  if (state.status === 'loading') return <p>Loading…</p>;
  if (state.status === 'error') return <p role="alert">Could not load user: {state.message}</p>;
  return <h1>{state.user.name}</h1>;
}

const json = (body, status = 200) => new Response(JSON.stringify(body), { status, headers: { 'Content-Type':
'application/json' } });
afterEach(() => { cleanup(); mock.restoreAll(); });    // put the real fetch back after every test

test('shows the user returned by the server, and calls the right URL', async () => {
  const fetchMock = mock.method(globalThis, 'fetch', async () => json({ id: 7, name: 'Ada Lovelace' }));
  render(<UserProfile userId={7} />);
  assert.equal((await screen.findByRole('heading')).textContent, 'Ada Lovelace');
  assert.equal(fetchMock.mock.callCount(), 1);
  assert.equal(fetchMock.mock.calls[0].arguments[0], '/api/users/7');
});

test('shows an error for an HTTP error status (fetch does not reject on 404)', async () => {
  mock.method(globalThis, 'fetch', async () => json({ error: 'not found' }, 404));
  render(<UserProfile userId={999} />);
  assert.equal((await screen.findByRole('alert')).textContent, 'Could not load user: Server said 404');
});

test('shows an error when the network fails', async () => {
  mock.method(globalThis, 'fetch', async () => { throw new TypeError('Failed to fetch'); });
  render(<UserProfile userId={1} />);
  assert.match((await screen.findByRole('alert')).textContent, /Failed to fetch/);
});

test('a slow old response cannot overwrite a newer one (race condition)', async () => {
  // user 1 answers after 100 ms, user 2 after 10 ms; honour the abort signal like real fetch does
  const delays = { '/api/users/1': 100, '/api/users/2': 10 };
  mock.method(globalThis, 'fetch', (url, { signal }) => new Promise((resolve, reject) => {
    const t = setTimeout(() => resolve(json({ name: `User from ${url}` })), delays[url]);
    signal.addEventListener('abort', () => { clearTimeout(t); reject(new DOMException('aborted', 'AbortError')); });
  }));
  const { rerender } = render(<UserProfile userId={1} />);
  rerender(<UserProfile userId={2} />);                    // user switches before the first answer
  assert.equal((await screen.findByRole('heading')).textContent, 'User from /api/users/2');
  await new Promise(r => setTimeout(r, 150));              // wait past the slow response
  assert.equal(screen.getByRole('heading').textContent, 'User from /api/users/2');
});
// ... plus a test that clicks "Next user" and waits for the second request's result
```

Output:

```
✔ shows the user returned by the server, and calls the right URL (67.915ms)
✔ shows an error for an HTTP error status (fetch does not reject on 404) (6.110375ms)
✔ shows an error when the network fails (4.473667ms)
✔ a slow old response cannot overwrite a newer one (race condition) (168.044584ms)
✔ mock only the network call, then drive the UI as a user (20.320625ms)
ℹ tests 5
ℹ pass 5
ℹ fail 0
```

Does the race test really test something? (A **race condition** is a bug where the result depends on which of two overlapping operations finishes first — here, two requests.) A test that can't fail is worthless. Removing the `controller.abort()` cleanup (the function an Effect returns, run before the next Effect or on unmount; `abort()` cancels the request still in flight) from `UserProfile` (a temporary copy of the file) makes it fail:

```
✖ a slow old response cannot overwrite a newer one (race condition) (168.159708ms)
  + actual - expected
    actual: 'User from /api/users/1',
    expected: 'User from /api/users/2',
```

Checking that a test fails when the code is broken is a good habit for any test you're not sure about. (The race itself and the `AbortController` fix are explained in Chapter 5.)

The last test shows the preferred style: mock **only the network boundary**, then drive the real UI with `user-event`.

Mock Service Worker (MSW) — the usual choice in real projects

MSW intercepts requests at the network level (in the browser through a Service Worker, a background script that sits between the page and the network) instead of replacing `fetch`, so it works for `fetch`, `axios` (a popular HTTP client library), TanStack Query (a library that fetches and caches server data), and anything else, and the same handlers (functions that decide the fake response for a URL) can power the dev server (the local server you run while developing) in the browser. Sketch (MSW 2 — not run here):

```
// test/server.js
import { setupServer } from 'msw/node';
import { http, HttpResponse } from 'msw';
export const server = setupServer(
  http.get('/api/users/:id', ({ params }) => HttpResponse.json({ id: params.id, name: 'Ada Lovelace' })),
);
// setupTests.js
beforeAll(() => server.listen());
afterEach(() => server.resetHandlers());       // undo per-test overrides
afterAll(() => server.close());
// in one test: server.use(http.get('/api/users/:id', () => new HttpResponse(null, { status: 404 })));
```

Approach	Good for	Drawback
Replace `fetch` (`mock.method`, `vi.spyOn`)	small projects, a few tests, no extra dependency	tied to `fetch`; each test repeats response setup
MSW	real apps; any HTTP client; shared handlers for tests, Storybook (a tool for viewing components in isolation) and dev	one more library and setup file
Mock your own API module (`vi.mock('./api')`)	quick isolation	skips your fetch/parse/error code — lower confidence

Components using **TanStack Query** also need a `QueryClientProvider` in the test wrapper — create a **new** `QueryClient` per test with `retry: false`, so tests don't share cache (stored results of earlier requests that get reused) and failures appear immediately (Chapter 9; providers in §12.8).

> **TL;DR**
>
> Mock at the network boundary, return real `Response` objects, test success, HTTP error (`res.ok` false), network error and races; restore mocks after each test. In real projects use MSW.

12.7 Testing custom Hooks

Two approaches

1. **Through a component** that uses the Hook the way the app does — the most realistic, and often all you need.
2. **`renderHook`** from RTL for reusable Hooks with a clear API (a library-style `useCounter`, `useDebounce`): it renders a hidden test component that calls your Hook and exposes the latest return value as `result.current`.

Hooks can't be called outside a component (Chapter 7), which is why both approaches render something.

`renderHook`, `act`, and `rerender`

- `const { result, rerender, unmount } = renderHook(() => useX(args), { initialProps, wrapper })`.
- Read `result.current` **after** each update — it's replaced on every render, so don't keep an old copy of it.
- Wrap calls that update state in **`act(() => ...)`**, because no RTL event helper is doing it for you.
- `rerender(newProps)` re-renders the same Hook instance with new arguments (use `initialProps` + a callback taking props).
- For Hooks that need context, pass `wrapper` exactly as in §12.8.

Fake timers

Time-based Hooks (debounce, throttle — run at most once per time interval — polling, timeouts) should not make tests actually wait. **Fake timers** replace `setTimeout` & co. with versions you advance manually. In `node:test`: `mock.timers.enable({ apis: ['setTimeout'] })` and `mock.timers.tick(ms)`; in Vitest `vi.useFakeTimers()` / `vi.advanceTimersByTime(ms)`; in Jest `jest.useFakeTimers()` / `jest.advanceTimersByTime(ms)`. Advance timers inside `act` so React processes the resulting updates.

`code/06_testing_hooks.jsx`:

```
function useCounter(initial = 0, { min = -Infinity, max = Infinity } = {}) {
  const [count, setCount] = useState(initial);
  const increment = useCallback(() => setCount(c => Math.min(max, c + 1)), [max]);
  const decrement = useCallback(() => setCount(c => Math.max(min, c - 1)), [min]);
  const reset = useCallback(() => setCount(initial), [initial]);
  return { count, increment, decrement, reset };
}
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}

afterEach(() => { cleanup(); mock.timers.reset(); });

test('useCounter: increments, respects max, resets', () => {
  const { result } = renderHook(() => useCounter(8, { max: 10 }));
  assert.equal(result.current.count, 8);
  act(() => result.current.increment());              // state updates must be wrapped in act()
  act(() => result.current.increment());
  act(() => result.current.increment());              // would be 11, capped at 10
  assert.equal(result.current.count, 10);
  act(() => result.current.reset());
  assert.equal(result.current.count, 8);
});

test('useCounter: rerender with new arguments', () => {
  const { result, rerender } = renderHook(({ max }) => useCounter(0, { max }), { initialProps: { max: 1 } });
  act(() => { result.current.increment(); result.current.increment(); });
  assert.equal(result.current.count, 1);
  rerender({ max: 5 });                               // same Hook instance, new props
  act(() => result.current.increment());
  assert.equal(result.current.count, 2);
});

test('useDebounce: only the last value within the delay gets through', () => {
  mock.timers.enable({ apis: ['setTimeout'] });
  const { result, rerender } = renderHook(({ v }) => useDebounce(v, 300), { initialProps: { v: 'r' } });
  rerender({ v: 're' });
  act(() => mock.timers.tick(200));
  rerender({ v: 'rea' });                             // restarts the 300 ms timer
  act(() => mock.timers.tick(299));
  assert.equal(result.current, 'r');                  // nothing has settled yet
  act(() => mock.timers.tick(1));
  assert.equal(result.current, 'rea');                // 're' was skipped entirely
});

function Stepper() {
  const { count, increment, decrement } = useCounter(0, { min: 0 });
  return <><button onClick={decrement}>-</button><output aria-label="count">{count}</output><button onClick=
{increment}>+</button></>;
}
test('useCounter through a component: cannot go below min', async () => {
  const user = userEvent.setup();
  render(<Stepper />);
  await user.click(screen.getByRole('button', { name: '-' }));
  await user.click(screen.getByRole('button', { name: '+' }));
  await user.click(screen.getByRole('button', { name: '+' }));
  assert.equal(screen.getByLabelText('count').textContent, '2');
});
```

Output:

```
✔ useCounter: increments, respects max, resets (7.892375ms)
✔ useCounter: rerender with new arguments (1.192583ms)
✔ useDebounce: only the last value within the delay gets through (1.022ms)
✔ useCounter through a component: cannot go below min (79.11725ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

The debounce test runs in about 1 ms even though it covers 800 ms of "time".

Common mistakes

- Destructuring once — `const { count } = result.current` — and expecting `count` to update. Read `result.current.count` again after each `act`.
- Calling the Hook directly (`useCounter()` outside a component) → "Invalid hook call".
- Leaving fake timers on after a test; reset them in `afterEach`. Also note that `user-event` waits between actions using timers — with fake timers in Vitest/Jest, create it with `userEvent.setup({ advanceTimers: vi.advanceTimersByTime })` (or `jest.advanceTimersByTime`).
- Importing `renderHook` from `@testing-library/react-hooks` — that separate package is for React 17 and older; since RTL 13.1, `renderHook` is part of `@testing-library/react`.

TL;DR

Test Hooks through a component when you can; otherwise `renderHook` + `act` + `result.current`, `rerender` for new args, fake timers for time.

12.8 Providers, routers and custom render

The problem it solves

Real components depend on providers (components that supply a shared value to everything inside them): context (theme, auth, cart), a router (the library that maps URLs to screens), a TanStack Query client, a Redux store (Redux is a state-management library that keeps app state in one central "store" object). Rendering such a component alone crashes. Repeating the provider tree in every test is noisy, so projects write one **custom** `render` that wraps the UI in everything the app needs, using RTL's `wrapper` option.

`code/07_providers_and_router.jsx`:

```
export function cartReducer(state, action) {            // pure function: unit-test it directly
  switch (action.type) {
    case 'add': return { ...state, [action.sku]: (state[action.sku] ?? 0) + 1 };
    case 'clear': return {};
    default: throw new Error(`unknown action ${action.type}`);
  }
}
const CartContext = createContext(null);
function CartProvider({ children, initial = {} }) {
  const [cart, dispatch] = useReducer(cartReducer, initial);
  return <CartContext value={{ cart, dispatch }}>{children}</CartContext>;
}
// CartBadge reads the cart; ProductPage reads :sku from the URL and dispatches 'add';
// App = <nav> with <Link>s and <CartBadge/> + <Routes> for "/" and "/p/:sku".

function renderWithProviders(ui, { route = '/', cart = {} } = {}) {
  const Wrapper = ({ children }) => (
    <MemoryRouter initialEntries={[route]}>            {/* a router that keeps history in memory */}
      <CartProvider initial={cart}>{children}</CartProvider>
    </MemoryRouter>
  );
  return { user: userEvent.setup(), ...render(ui, { wrapper: Wrapper }) };
}

test('reducer: plain unit test, no React needed', () => {
  assert.deepEqual(cartReducer({ MUG: 1 }, { type: 'add', sku: 'MUG' }), { MUG: 2 });
  assert.deepEqual(cartReducer({ MUG: 1 }, { type: 'clear' }), {});
  assert.throws(() => cartReducer({}, { type: 'oops' }), /unknown action oops/);
});

test('starts on a given route with a pre-filled cart', () => {
  renderWithProviders(<App />, { route: '/p/LAMP', cart: { MUG: 2 } });
  assert.equal(screen.getByRole('heading').textContent, 'Product LAMP');
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});

test('navigates with a link, then adds to cart', async () => {
  const { user } = renderWithProviders(<App />);
  assert.equal(screen.getByRole('heading').textContent, 'Home');
  await user.click(screen.getByRole('link', { name: 'Mug' }));
  assert.equal(screen.getByRole('heading').textContent, 'Product MUG');
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  await user.click(screen.getByRole('button', { name: 'Add to cart' }));
  assert.equal(screen.getByLabelText('items in cart').textContent, '2');
});

test('without the provider the component crashes - which is why the wrapper exists', () => {
  const quiet = console.error; console.error = () => {};
  assert.throws(() => render(<CartBadge />), /Cannot destructure property 'cart'/);
  console.error = quiet;
});
```

Output:

```
✔ reducer: plain unit test, no React needed (0.762041ms)
✔ starts on a given route with a pre-filled cart (58.361708ms)
✔ navigates with a link, then adds to cart (45.55225ms)
✔ without the provider the component crashes - which is why the wrapper exists (2.687916ms)
ℹ tests 4
ℹ pass 4
ℹ fail 0
```

How it works

1. `render(ui, { wrapper: Wrapper })` renders `<Wrapper>{ui}</Wrapper>`; `rerender` keeps the wrapper.
2. `MemoryRouter` (from `react-router`) keeps navigation history in memory instead of the address bar; `initialEntries` sets the starting URL. For apps built with data routers (`createBrowserRouter`,

loaders/actions — functions a route runs to load data before rendering, or to handle a form submission), use `createMemoryRouter(routes, { initialEntries })` + `<RouterProvider>` in tests.

3. Options such as `route` and `cart` let each test start from a specific state **through the public API** (props of the providers), not by poking internal state.
4. Put `renderWithProviders` in a shared `test-utils` file and import it instead of RTL's `render`. With Redux Toolkit (the official, simpler way to write Redux), create a **fresh store per test** in the wrapper; with TanStack Query, a **fresh** `QueryClient` — shared instances leak state between tests.

Unit tests still have a place

The reducer is a pure function, so it gets a plain, very fast unit test — no rendering. Test pure logic (reducers, formatters, validation schemas) directly, and test **components** through the UI.

> **TL;DR**
>
> One `renderWithProviders` with a `wrapper` (router with `initialEntries`, context, store/query client created fresh per test). Unit-test pure functions directly.

12.9 Setting up Vitest or Jest in a real project

The tests in this chapter run with `node:test` so they need no extra tooling. Real React projects usually use **Vitest** (natural for Vite projects; Jest-compatible API) or **Jest**. The component tests themselves look the same — only imports, `expect` and mock helpers differ. The configurations below are standard setups, **not run in this book**.

Vitest (Vite projects)

```
npm install -D vitest jsdom @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// vite.config.js (or vitest.config.js)
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';
export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',          // simulated browser DOM (alternative: 'happy-dom')
    globals: true,                 // describe/test/expect/afterEach as globals -> RTL auto-cleanup works
    setupFiles: './src/setupTests.js',
  },
});
```

```
// src/setupTests.js
import '@testing-library/jest-dom/vitest';   // adds toBeInTheDocument(), toBeDisabled(), toHaveTextContent()...
```

```
// package.json
{ "scripts": { "test": "vitest", "test:run": "vitest run", "coverage": "vitest run --coverage" } }
```

A test file (`Counter.test.jsx`):

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { Counter } from './Counter';

test('increments', async () => {
  const user = userEvent.setup();
  render(<Counter />);
  await user.click(screen.getByRole('button', { name: /increment/i }));
  expect(screen.getByText('Count: 1')).toBeInTheDocument();
});
```

With TypeScript, add `"types": ["vitest/globals", "@testing-library/jest-dom"]` to `tsconfig.json` so `test`/`expect` and the jest-dom matchers are typed. Without `globals: true`, import `test`, `expect`, `afterEach` from `vitest` and call `afterEach(cleanup)` yourself.

Jest

```
npm install -D jest jest-environment-jsdom babel-jest @babel/preset-env @babel/preset-react \
  @testing-library/react @testing-library/dom @testing-library/user-event @testing-library/jest-dom
```

```
// jest.config.js
module.exports = {
  testEnvironment: 'jsdom',                       // needs jest-environment-jsdom (separate since Jest 28)
  setupFilesAfterEnv: ['<rootDir>/jest.setup.js'], // runs after the test framework is installed
  moduleNameMapper: { '\\.(css|less|svg)$': '<rootDir>/test/fileMock.js' },  // stub non-JS imports
};
// jest.setup.js
import '@testing-library/jest-dom';
// babel.config.js
module.exports = { presets: ['@babel/preset-env', ['@babel/preset-react', { runtime: 'automatic' }]] };
```

TypeScript with Jest uses `ts-jest` or Babel's TypeScript preset (Babel is a tool that translates JSX, TypeScript and modern JavaScript into plain JavaScript). Vitest needs none of this because it reuses Vite's transform pipeline (the chain of steps Vite already uses to turn your source files into runnable JavaScript) — one reason new Vite projects usually choose it.

Useful extras

Tool	For
`@testing-library/jest-dom`	readable DOM matchers
MSW	network mocking (§12.6)
`vitest --coverage` / `jest --coverage`	coverage: which lines ran — a guide to untested code, not a goal in itself
`eslint-plugin-testing-library`, `eslint-plugin-jest-dom`	lint rules (automatic code checks) that catch misuse (e.g. `getBy` inside `waitFor`, missing `await`)
Vitest Browser Mode, Playwright component testing	running component tests in a real browser when jsdom isn't enough (layout, real CSS, canvas)
Playwright / Cypress	end-to-end tests

What jsdom can't do

jsdom implements the DOM and events but **no layout or painting** (the browser's work of calculating sizes and positions, then drawing pixels): sizes and positions are 0, `scrollTop` changes only when you set it (as in Chapter 10's virtualization demo — rendering only the visible rows of a long list), CSS media queries (styles that apply only at certain screen sizes) don't apply, `IntersectionObserver` (reports when an element scrolls into view), `ResizeObserver` (reports size changes) and `matchMedia` (checks a media query from JavaScript) aren't there by default and must be mocked. Anything that depends on real rendering belongs in browser-based or E2E tests.

> **TL;DR**

Vitest: `environment: 'jsdom'`, `globals: true`, a setup file importing jest-dom. Jest: `jest-environment-jsdom`, Babel/TS transform, `setupFilesAfterEnv`. Same RTL tests either way.

12.10 Good tests, bad tests, and the bigger picture

What makes a good component test

1. **Named after behaviour**: "shows an error when the server returns 404", not "test 3" or "sets error state".
2. **Arrange–act–assert**, one behaviour per test (several `assert`s are fine if they describe one behaviour).
3. **Uses the public interface**: props in, DOM out, user events in between, network mocked at the boundary.
4. **Independent**: no shared DOM, mocks, stores or caches between tests; any order works.
5. **Deterministic** (same result every run): no real network, no real time, no randomness (or seeded, i.e. random numbers from a fixed starting value so they repeat); waits for conditions, never sleeps.
6. **Fails for the right reason**: break the feature and the test goes red with a clear message (§12.6's race test).

Brittle tests to avoid

A **brittle** test breaks when the code changes in harmless ways, even though the feature still works.

Brittle	Why	Instead
Asserting on state, props, or Hook internals	breaks on refactor, passes when UI is broken	assert on rendered output
`container.querySelector('.css-1x2y3z')`	class names are implementation details	role/label/text queries
Large **snapshot** tests (saved copy of the rendered output; fails on any change) of whole components	any markup change fails; reviewers approve new snapshots without reading them	explicit assertions on what matters; small inline snapshots at most
Mocking child components	the test no longer checks real integration	render the real tree; mock only the network or truly heavy leaf modules
Asserting that a handler was called with internal arguments	couples to wiring	assert the visible effect of the handler
Testing third-party libraries (code written by others that you install)	not your code	test your usage of them via the UI

Real-world examples

- A checkout form: tests for "button disabled until valid", "shows field errors", "submits the right payload" (the data sent in the request; checked at the mocked network), "shows server error", plus one E2E test of the full purchase in a real browser.
- A design-system `Dialog`: role `dialog`, labelled by its title, focus moves inside on open, Escape closes, focus returns to the trigger (the button that opened it) — all testable with RTL + user-event, and each one is also an accessibility requirement (Chapter 14).

In the interview

30-second answer: "I use React Testing Library with Vitest or Jest. Tests render the component, find elements the way users do — by role and accessible name first — interact with user-event, and assert on the DOM, not on state or internals. Async UI is awaited with `findBy` or `waitFor`, never sleeps. The network is mocked at the boundary, ideally with MSW, covering success, errors and races. Hooks are tested through components or `renderHook` with `act`, and components that need providers use a custom render with a wrapper. E2E with Playwright covers the few critical flows."

- *`getBy` vs `queryBy` vs `findBy`?* — `getBy` throws if missing (must exist now), `queryBy` returns `null` (to assert absence), `findBy` returns a promise that retries (for async appearance).
- *Why prefer `getByRole`?* — It matches how users and assistive technology find elements, doubles as an accessibility check, and survives markup refactors.
- *What does `act` do and when do you need it?* — It flushes (applies right away) React updates and effects before assertions; RTL wraps its helpers in it, so you need it only for direct updates such as `renderHook` calls or advancing fake timers.
- *How do you test a component that fetches?* — Mock the network (MSW or a `fetch` spy), `await findBy…` the loaded UI, and test error and loading states too.

Self-check

1. Which query would you use to find a "Save" button? → `screen.getByRole('button', { name: 'Save' })`.
2. How do you assert an error message is **not** shown? → `assert.equal(screen.queryByRole('alert'), null)` (or `expect(...).not.toBeInTheDocument()`).
3. Why is `await new Promise(r => setTimeout(r, 500))` a bad way to wait for data? → It's slow and flaky; wait for the condition with `findBy`/`waitFor`.
4. How do you test a `useDebounce` Hook without waiting? → Fake timers, advanced inside `act`.
5. Why does the custom render create a fresh store/query client per test? → Shared instances leak state between tests and make them order-dependent.
6. Why must `node:test` files call `afterEach(cleanup)`? → RTL auto-cleans only when a global `afterEach` exists (Jest, Vitest with globals).

Chapter 12 TL;DR

Idea	Remember
Philosophy	Test what users see and do; ignore implementation details
Basic test	`render` → `screen` query → interact → assert → `cleanup`
Query priority	Role (+ name) → label → placeholder → text → display value → alt → title → test ID
Variants	`get` now / `query` maybe-absent / `find` wait; `All` for many; `within` to scope
Interactions	`const user = userEvent.setup(); await user.click/type/selectOptions/keyboard/tab(...)`
Async	`findBy`, `waitFor(() => assertion)`, `waitForElementToBeRemoved`; never sleep
Network	Mock at the boundary (MSW, or a `fetch` spy returning `Response`s); cover success, HTTP error, network error, races
Hooks	Through a component, or `renderHook` + `act` + `result.current`; fake timers for time
Providers	Custom render with `wrapper`: `MemoryRouter`, context, fresh store/query client per test
Setup	Vitest (`jsdom`, `globals`, jest-dom setup) or Jest (`jest-environment-jsdom`, Babel/TS)
Quality	Behaviour-named, independent, deterministic, fails for the right reason; few snapshots; E2E for critical paths

Next: Chapter 13 — Server Rendering and Frameworks.